

## Regression vs Classification

- Regression -> Continuous
- Classification -> Discrete (Labels & Categorical)

Regression and Classification is differentiated by it's dependent variable.

## 2. Load libraries & data

```
library(rpart)
library(rpart.plot)
library(MLmetrics)

# Regression use RMSE & Rsquared, pROC -> ROC
# Classification Sensivity, Specificity, Accuracy

autompkg <- read.table("auto-mpg.data")
colnames(autompkg) <- c("mpg", "cylinders", "displacement", "horsepower",
"weight", "acceleration", "model_year", "origin", "car_name")
head(autompkg)
```

## 3. Statistics Summary

```
basicSummary <- function(df, dgts = 3) {
  m <- ncol(df) # column
  varNames <- colnames(df)
  varType <- vector("character", m)
  topLevel <- vector("character", m)
  topCount <- vector("numeric", m)
  missCount <- vector("numeric", m)
  levels <- vector("numeric", m)

  for (i in 1:m) {
    x <- df[,i]
    varType[i] <- class(x)
    xtab <- table(x, useNA = "ifany")
    levels[i] <- length(xtab)
    nums <- as.numeric(xtab)
    maxnum <- max(nums)
    topCount[i] <- maxnum
    maxIndex <- which.max(nums)
    lvls <- names(xtab)
    topLevel[i] <- lvls[maxIndex]
    missIndex <- which((is.na(x)) | (x == "") | (x == " "))
    missCount[i] <- length(missIndex)
  }
  n <- nrow(df) # Rows
  topFrac <- round(topCount/n, digits = dgts)
  missFrac <- round(missCount/n, digits = dgts)
  summaryFrame <- data.frame(variable = varNames,
```

```

        type = varType,
        levels = levels,
        topLevel = topLevel,
        topCount = topCount,
        topFrac = topFrac,
        missFreq = missCount,
        missFrac = missFrac)

return(summaryFrame)
}

```

#### 4. Data Preparation

In Summary : Data Problems.

1. car\_name is meaningless -> "remove"
2. convert all var into numerical
3. Horsepower have missng value

Rule of thumb : if missing value < 20% of total data then we can remove it, if not then median is recommended

```

# Remove car_name
autmpg <- subset(autmpg, select = -c(car_name))

# Change all var into numerical
autmpg$horsepower <- as.numeric(autmpg$horsepower)

## Warning: NAs introduced by coercion

autmpg$cylinders <- as.numeric(autmpg$cylinders)
autmpg$model_year <- as.numeric(autmpg$model_year)
autmpg$origin <- as.numeric(autmpg$origin)

# Change missing value with mean
autmpg$horsepower[is.na(autmpg$horsepower)] <- mean(autmpg$horsepower,
na.rm = T)
basicSummary(autmpg)

```

Warning -> because some of the character strings aren't properly structures integers and so can't be translated to numeric

#### 5. Split data

```

set.seed(123)
n <- nrow(autmpg)
train <- sample(n, round(0.7 * n))
autoTrain <- autmpg[train,]
autoTest <- autmpg[-train,]

```

## 6. Fit the Model

### 6A. Linear Regression Models

```
LRModel1 <- lm(mpg ~., autoTrain) # Model 1  
#summary(LRModel1)
```

```
LRModel2 <- lm(mpg ~ weight + model_year + origin, autoTrain) # Model2
```

### 6B. Decision Tree Models

```
DTModel1 <- rpart(mpg ~ ., autoTrain) # Model 1  
#DTModel1$variable.importance  
DTModel2 <- rpart(mpg ~ displacement + weight + cylinders + horsepower,  
autoTrain) # Model 2
```

Getting top 4 for the 2nd Model

```
rpart.plot(DTModel1)  
rpart.plot(DTModel2)
```

## 7. Model Validation

```
predLR1 <- predict(LRModel1, autoTest)  
predLR2 <- predict(LRModel2, autoTest)  
predDT1 <- predict(DTModel1, autoTest)  
predDT2 <- predict(DTModel2, autoTest)
```

## 8. Model Performance

```
# Calculating RMSE  
valLR1 <- RMSE(predLR1, autoTest$mpg)  
valLR2 <- RMSE(predLR2, autoTest$mpg)  
valDT1 <- RMSE(predDT1, autoTest$mpg)  
valDT2 <- RMSE(predDT2, autoTest$mpg)  
  
# Calculationg R2_Score  
valLR1 <- R2_Score(predLR1, autoTest$mpg)  
valLR2 <- R2_Score(predLR2, autoTest$mpg)  
valDT1 <- R2_Score(predDT1, autoTest$mpg)  
valDT2 <- R2_Score(predDT2, autoTest$mpg)
```

## 9. Conclusion

We can conclude that the best model is the Linear Regression Model 2 because it has the least RMSE and the highest R2\_Score

Classification The same as above but with a few twist

## Train a Decision Tree

### Fitting the model with all of the variables

```
medvModel1 <- rpart(formula = medv ~.,
                    data = bostonTrain,
                    method = "class")
medvModel1$variable.importance

##      lstat      rm      nox      age      indus      dis      ptratio
## 88.200960 61.049274 50.220325 49.841110 47.869093 45.505345 6.362580
## 5.850207
##      rad      tax
## 3.045725 2.280022

library(rpart.plot)
rpart.plot(x = medvModel1, yesno = 2, type = 0)
```

### Fit the model only with top 3 most important variables

```
medvModel2 <- rpart(formula = medv ~ lstat + rm + nox,
                    data = bostonTrain,
                    method = "class")
medvModel2$variable.importance

##      lstat      rm      nox
## 91.93587 62.17331 57.13765
```

## Model Validation

```
predModel1 <- predict(medvModel1, newdata = bostonTest, type = "class")
predModel2 <- predict(medvModel2, newdata = bostonTest, type = "class")
```

## Model Performance

```
confusionMatrix <- function(Model, testData) {
  cm <- table(Model, testData)

  accuracy <- sum(cm[1], cm[4]) / sum(cm[1:4])
  precision <- cm[4] / sum(cm[4], cm[2])
  sensitivity <- cm[4] / sum(cm[4], cm[3])
  fscore <- (2 * (sensitivity * precision)) / (sensitivity + precision)
  specificity <- cm[1] / sum(cm[1], cm[2])
  sprintf("Accuracy is %s", round(accuracy, 3))
}
```

## EDA

- We mostly use sample to be analyzed.
- Confidence Level : 1-alpha (mostly used : 90%, 95%, 99%)

- Level of significance : the chances of the data is real and not just a lucky chance.
- $p\text{-value} \leq 0.05 \rightarrow$  Independent variable is statistically significant with the dependent variable
- Recommended to use confidence interval : Search for lower limit and upper limit.
- If we have a binary variable then check for binomial distribution.
- If we have count variable then check for poisson distribution.
- If we have a discrete or continuous then check for normal distribution (QQ-plot).
- If 2 of numerical variable then search for coefficient of correlation 1 being the best
- Pearson is for Interval (Nominal)
- Spearman is for Ordinal
- We can search for correlation of categorical we can use chi-squared

## EDA Steps :

- Search for Descriptive Statistics  $\rightarrow$  use the summary() Function
- Do Bivariate Analysis (2 Variable)  $\rightarrow$  plot and friends -
- Do Multivariate Analysis (corr coefficient)

## Anomalies

User Defined Function to Calculate How many anomalies

```
extreme_values <- function(data) {
  # Calculate the IQR of the variable
  iqr <- IQR(data, na.rm = TRUE)

  # Calculate the lower and upper bounds for outliers
  lower_bound <- quantile(data, 0.25, na.rm = TRUE) - 1.5 * iqr
  upper_bound <- quantile(data, 0.75, na.rm = TRUE) + 1.5 * iqr

  # Count the number of extreme values
  extreme <- data[data < lower_bound | data > upper_bound]

  return(extreme)
}
```